

TAPSI

Restaurant Management System

for Tapsilogans, Karinderyas, and Small Philippine Food Businesses

SOFTWARE REQUIREMENTS SPECIFICATION

Version 2.0 (Complete)

Prepared by: Benjie Edroso

Date: July 18, 2026

Status: Baseline for Development

Revision History

Version	Date	Author	Description
0.9	2026-07-10	Benjie Edroso	Initial outline and module list
1.0	2026-07-15	Benjie Edroso	First SRS draft: modules, roles, tech stack, roadmap
2.0	2026-07-18	Benjie Edroso	Complete specification: detailed functional requirements with IDs, user stories with acceptance criteria, use cases, permission matrix, NFRs, business rules, database data dictionary, API specification, UI/UX, security, Microsoft Azure architecture, DR, observability, testing, appendices

Approval

Role	Name	Signature	Date
Product Owner	Benjie Edroso		
Lead Developer			
QA Lead			

Table of Contents

Right-click and choose 'Update Field' to refresh.

Note: open in Microsoft Word and press Ctrl+A then F9 (or right-click → Update Field) to populate the table of contents and page numbers.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for TAPSI, a cloud-based Restaurant Management System (RMS) designed for small food businesses in the Philippines, including tapsilogans, karinderyas, eateries, paresan, milk tea shops, and cafeterias. This document serves as the single source of truth for developers, testers, designers, and stakeholders during the design, implementation, verification, and deployment of the system.

This specification is intended to be sufficiently detailed to allow a development team to implement the system without requiring further requirements elicitation, and to allow a QA team to derive complete test cases from it.

1.2 Product Scope

TAPSI replaces manual notebooks, spreadsheets, and disconnected point-of-sale tools with a single centralized, multi-tenant SaaS platform. Version 1.0 covers:

- User authentication, role-based authorization, and audit logging
- Menu and category management with images and add-ons
- Inventory management with immutable transaction history
- Supplier and purchase order management with automatic stock updates
- Recipe management with automatic ingredient deduction on sale
- Order management for dine-in, take-out, and delivery workflows
- Table management (status, transfer, merge)
- Payment processing (Cash, GCash, Card, Bank Transfer, split payments) and receipt generation
- Expense tracking with categories and approvals
- Employee profiles and attendance
- Operational and financial reporting with CSV/PDF export
- Daily closing (end-of-day cash reconciliation) with day locking

- In-app notifications

Out of scope for Version 1.0: mobile applications, QR ordering, third-party delivery integrations (GrabFood, foodpanda), loyalty programs, payroll computation, multi-branch consolidation, and offline mode. These are listed in Section 17 (Future Enhancements).

1.3 Intended Audience and Reading Suggestions

Audience	Relevant Sections	Purpose
Restaurant Owners / Stakeholders	1, 2, 3, 5, 8, 17	Validate that the system meets business needs
Product Manager	All	Scope control, backlog derivation, release planning
Backend Developers	4, 6, 7, 8, 9, 10, 12	Implement APIs, business logic, and database
Frontend Developers	4, 5, 6, 10, 11	Implement screens and client-side workflows
QA Engineers	4, 5, 6, 7, 8, 16	Derive test plans, test cases, and acceptance tests
DevOps Engineers	7, 13, 14, 15	Provision infrastructure, CI/CD, monitoring
Security Reviewers	7, 12, 15	Assess security posture and compliance

1.4 Definitions, Acronyms, and Abbreviations

Key terms are defined in Appendix A (Glossary). The following conventions are used throughout this document:

- FR-xxx — Functional Requirement identifier
- NFR-xxx — Non-Functional Requirement identifier
- BR-xxx — Business Rule identifier
- US-xxx — User Story identifier
- UC-xx — Use Case identifier
- "Shall" denotes a mandatory requirement; "should" denotes a recommended requirement; "may" denotes an optional requirement.
- Priorities follow MoSCoW: M (Must have), S (Should have), C (Could have), W (Won't have in v1.0).

1.5 References

Ref	Document / Standard	Relevance
R1	IEEE 830-1998 / ISO/IEC/IEEE 29148:2018	SRS structure and quality guidance
R2	Republic Act 10173 — Data Privacy Act of 2012 (PH)	Handling of personal data of employees and users
R3	Republic Act 9994 (Expanded Senior Citizens Act) and RA 10754 (PWD)	Mandatory 20% discount and VAT exemption rules
R4	BIR Revenue Regulations on Official Receipts / Sales Invoices	Receipt content requirements
R5	OWASP Application Security Verification Standard (ASVS) 4.0	Security requirements baseline
R6	Django 5.x, Django REST Framework 3.x documentation	Implementation framework
R7	PostgreSQL 16 documentation	Database platform

1.6 Document Overview

Section 2 describes the product context, users, and constraints. Section 3 defines roles and the permission matrix. Section 4 specifies functional requirements per module. Section 5 contains user stories with acceptance criteria. Section 6 contains detailed use case specifications. Sections 7–8 define non-functional requirements and business rules. Sections 9–10 specify the database design and REST API. Sections 11–16 cover UI/UX, security, cloud architecture, disaster recovery, observability, and testing. Section 17 lists future enhancements, followed by appendices.

2. Overall Description

2.1 Product Perspective

TAPSI is a new, self-contained multi-tenant SaaS product. Each restaurant (tenant) owns its data; tenants are logically isolated at the application and database layer via a restaurant_id foreign key scoped on every tenant-owned table and enforced in every query through a tenant-aware base queryset. The system consists of:

- A React + Vite single-page application (SPA) served via Azure Front Door/CDN
- A Django REST Framework JSON API served from Docker containers on Azure App Service behind Azure Front Door
- A PostgreSQL database hosted on Azure Database for PostgreSQL
- Azure Blob Storage for menu images, expense receipt images, and report exports
- Background workers (Celery + Redis) for notifications, report generation, and scheduled jobs

2.2 Product Functions (Summary)

#	Function	Description
1	Authentication & Profiles	JWT login/logout, password reset, profile management
2	Restaurant & User Management	Tenant registration, staff accounts, role assignment
3	Dashboard	Real-time sales, orders, low stock, and profit overview
4	Menu Management	Categories, menu items, add-ons, images, availability
5	Inventory Management	Ingredients, stock levels, adjustments, spoilage
6	Supplier Management	Supplier records, purchase history, balances
7	Purchase Orders	PO lifecycle with automatic stock receiving
8	Recipe Management	Bill of materials per menu item; auto-deduction
9	Order Management	Dine-in/take-out/delivery order lifecycle
10	Table Management	Table status, assignment, transfer, merge
11	Payments & Receipts	Multi-method and split payments, receipts, refunds
12	Expense Management	Operating expense capture with attachments
13	Employee Management	Profiles, attendance, linkage to user accounts
14	Reports	Sales, profit & loss, inventory, tax, exports
15	Daily Closing	Cash count, variance, day locking, EOD report
16	Notifications	Low stock, order events, closing reminders
17	Audit Logs	Immutable record of user and system actions

2.3 User Classes and Characteristics

User Class	Technical Skill	Frequency of Use	Description
Owner	Low–Medium	Daily	Full access; manages subscription, users, daily closing, and all reports
Manager	Medium	Daily	Operations: menu, inventory, orders, employees, reports; no subscription or user deletion
Cashier	Low	Continuous during shift	Creates orders, accepts payments, prints receipts
Kitchen Staff	Low	Continuous during shift	Views order queue and updates preparation status
System Administrator (internal)	High	Occasional	Platform-level support and maintenance (not a tenant role)

2.4 Operating Environment

- Client: Latest two major versions of Chrome, Edge, Firefox, and Safari on desktop; Chrome on Android tablets (min 768px width)
- Server: Docker containers (Python 3.12, Django 5.x) on Azure App Service (Linux container)
- Database: PostgreSQL 16 on Azure Database for PostgreSQL (zone-redundant)
- Object storage: Azure Blob Storage (Southeast Asia region)
- Network: HTTPS only (TLS 1.2+); HTTP requests are redirected

2.5 Design and Implementation Constraints

- Backend must use Python, Django, and Django REST Framework; database must be PostgreSQL
- Frontend must use React with Vite
- Authentication must use JWT (short-lived access token + refresh token rotation)
- All infrastructure must run on Microsoft Azure; deployments via GitHub Actions CI/CD
- Monetary values must be stored as NUMERIC(12,2) in PHP (Philippine Peso); no floating point arithmetic for money
- All timestamps stored in UTC; displayed in Asia/Manila timezone
- The system must comply with the Data Privacy Act of 2012 for personal data
- Receipts must contain the fields required for BIR-compliant sales invoices (see FR-103)

2.6 Assumptions and Dependencies

- Each restaurant has a stable internet connection; offline mode is not supported in v1.0
- Each tenant operates a single branch in v1.0
- GCash and card payments are settled on external devices/apps; TAPSI records the payment method and reference number but does not process funds in v1.0
- Receipt printing uses the browser print dialog with a 58mm/80mm thermal-printer-friendly stylesheet
- Email delivery (password reset) depends on Azure Communication Services Email

3. User Roles and Permissions

3.1 Role Definitions

TAPSI implements role-based access control (RBAC). Every user belongs to exactly one restaurant and holds exactly one role. Permissions are enforced server-side on every endpoint; the UI additionally hides actions the current role cannot perform.

3.2 Permission Matrix

Legend: F = Full (create/read/update/delete), C = Create/Update, R = Read only, — = No access. Server-side enforcement is mandatory (see NFR-020).

Module / Capability	Owner	Manager	Cashier	Kitchen
Dashboard	F	F	R (sales widgets hidden)	—
Menu & Categories	F	F	R	R
Recipes	F	F	—	R
Inventory (ingredients, stock)	F	F	R	R

Module / Capability	Owner	Manager	Cashier	Kitchen
Inventory Adjustments / Spoilage	F	F	—	—
Suppliers	F	F	—	—
Purchase Orders	F	F	—	—
Orders — create/modify	F	F	C	—
Orders — cancel	F	F	Request only	—
Orders — update kitchen status	F	F	—	C
Tables	F	F	C (status/assign)	R
Payments — accept	F	F	C	—
Payments — refund/void	F	F	—	—
Expenses	F	F	C (create only)	—
Employees & Attendance	F	F	R (own record)	R (own record)
Reports	F	F	R (own shift X-reading)	—
Daily Closing	F	F (perform)	—	—
Reopen a closed day	F	—	—	—
Users — create/edit	F	C (Cashier/Kitchen only)	—	—
Users — delete/deactivate	F	—	—	—
Restaurant settings & subscription	F	—	—	—
Audit Logs	F	R	—	—
Notifications (own)	F	F	F	F

3.3 Authorization Rules

- A user shall never be able to read or mutate data belonging to another restaurant (tenant isolation).
- Role changes take effect on the user's next token refresh (at most 15 minutes) and immediately upon re-login.
- Deactivated users are rejected at authentication and token refresh.
- Permission checks are enforced in the API layer (DRF permission classes); UI visibility is a convenience, not a control.

4. Functional Requirements

Each requirement is uniquely identified (FR-xxx), testable, and prioritized using MoSCoW (M = Must, S = Should, C = Could). Unless stated otherwise, every create, update, and delete operation in this section shall (a) be scoped to the authenticated user's restaurant, (b) be permission-checked per Section 3.2, and (c) produce an audit log entry per Module 17.

4.1 Module 1: Authentication & User Profile

Provides secure access to the system using JWT-based authentication.

ID	Requirement	Priority
FR-001	The system shall authenticate users with email and password and, on success, issue a short-lived JWT access token (15 min) and a refresh token (7 days) with rotation.	M
FR-002	The system shall reject login attempts with invalid credentials with a generic error message that does not reveal whether the email exists.	M
FR-003	The system shall lock an account for 15 minutes after 5 consecutive failed login attempts and record the lockout in the audit log.	M
FR-004	The system shall allow a logged-in user to log out; the refresh token shall be blacklisted	M

ID	Requirement	Priority
	immediately.	
FR-005	The system shall provide password reset via a single-use, signed email link that expires after 1 hour.	M
FR-006	The system shall allow a logged-in user to change their password by providing the current password; all other active sessions of that user shall be invalidated.	M
FR-007	The system shall enforce password policy: minimum 10 characters, at least one letter and one number; passwords are checked against a common-password denylist.	M
FR-008	The system shall allow users to view and update their own profile (name, phone, avatar); email changes require re-verification.	S
FR-009	The system shall record every successful and failed login with timestamp, IP address, and user agent.	M

4.2 Module 2: Restaurant & User Management

Manages tenant (restaurant) registration, configuration, and staff accounts.

ID	Requirement	Priority
FR-010	The system shall allow a new restaurant to register with restaurant name, owner name, email, and password; registration creates the tenant and its Owner user atomically.	M
FR-011	The system shall allow the Owner to maintain restaurant settings: business name, address, contact number, TIN, VAT registration flag, logo, receipt footer message, currency (fixed PHP in v1.0), and timezone (fixed Asia/Manila in v1.0).	M
FR-012	The system shall allow Owners (any role) and Managers (Cashier and Kitchen roles only) to create staff user accounts with name, email, temporary password, and role.	M
FR-013	The system shall force staff users to change their temporary password on first login.	S
FR-014	The system shall allow the Owner to change any user's role; a user cannot change their own role.	M
FR-015	Only the Owner shall be able to deactivate or delete users; deletion is a soft delete that preserves historical references (orders, audit logs).	M
FR-016	The system shall prevent deletion or deactivation of the last active Owner account of a restaurant.	M
FR-017	The system shall isolate all tenant data: every API response shall contain only records belonging to the caller's restaurant.	M

4.3 Module 3: Dashboard

A role-aware landing page summarizing current business health.

ID	Requirement	Priority
FR-020	The dashboard shall display today's gross sales, net sales, order count, and average order value, computed for the restaurant's business day (Asia/Manila).	M
FR-021	The dashboard shall display current order counts by status (Pending, Preparing, Ready, Completed, Cancelled).	M
FR-022	The dashboard shall display the top 5 selling menu items for today and for the current month, by quantity and by revenue.	M
FR-023	The dashboard shall list ingredients at or below minimum stock with current quantity and unit.	M
FR-024	The dashboard shall display a 30-day revenue trend chart and a current-month revenue vs. expenses vs. profit summary.	S
FR-025	Dashboard widgets shall refresh automatically at most every 60 seconds without a full page reload.	S
FR-026	Cashiers shall see only the order-status and low-stock widgets; financial widgets shall be hidden (per Section 3.2).	M

4.4 Module 4: Menu Management

Maintains sellable items grouped by category, with images and add-ons.

ID	Requirement	Priority
FR-030	The system shall support category CRUD (name, display order, active flag); category names are unique per restaurant.	M
FR-031	The system shall support menu item CRUD with: name, description, category, price (PHP $\geq$ 0.00), availability flag, preparation time (minutes), and image.	M
FR-032	Menu images shall be uploaded to Blob Storage; accepted formats JPEG/PNG/WebP, max 5 MB; the system shall generate a 400px thumbnail.	M
FR-033	The system shall support add-ons (e.g., extra rice, extra egg) with their own price and optional recipe, attachable to one or more menu items.	S
FR-034	Toggling availability shall immediately hide/show the item on the order screen without deleting it.	M
FR-035	Menu item deletion shall be a soft delete; historical orders and reports shall continue to show the item's name and price at time of sale.	M
FR-036	The system shall retain a price change history (old price, new price, changed by, timestamp).	S
FR-037	The order screen shall support menu search by name and filtering by category.	M

4.5 Module 5: Inventory Management

Tracks ingredient stock with a complete, immutable transaction ledger.

ID	Requirement	Priority
FR-040	The system shall support ingredient CRUD with: name, unit of measure (g, kg, ml, L, pc, pack), current stock (derived), minimum stock, average unit cost, and default supplier.	M
FR-041	Current stock shall be derived from the sum of inventory transactions; direct edits to the stock figure shall not be possible.	M
FR-042	The system shall support transaction types: PURCHASE (in), CONSUMPTION (out, from recipes), ADJUSTMENT (in/out with mandatory reason), SPOILAGE (out with mandatory reason), and RETURN (out to supplier).	M
FR-043	Every inventory transaction shall record: ingredient, type, signed quantity, unit cost (where applicable), resulting balance, reference (e.g., order or PO id), user, and timestamp; transactions are immutable — corrections are made by compensating entries.	M
FR-044	The system shall reject any transaction that would make an ingredient's stock negative (see BR-001) and return a descriptive error identifying the ingredient and shortfall.	M
FR-045	When an ingredient's stock falls to or below its minimum stock, the system shall create a low-stock notification (once per ingredient until stock is replenished above the threshold).	M
FR-046	The system shall display per-ingredient stock card: running balance with all transactions, filterable by date range and type.	M
FR-047	Average unit cost shall be maintained using the weighted average cost method, recalculated on every PURCHASE transaction.	S

4.6 Module 6: Supplier Management

Maintains the vendors from whom the restaurant purchases ingredients.

ID	Requirement	Priority
FR-050	The system shall support supplier CRUD with: name, contact person, phone, email, address, payment terms (COD / 7 / 15 / 30 days), and active flag.	M
FR-051	The supplier detail view shall show purchase history (POs with dates, totals, status) and computed outstanding balance (received but unpaid POs).	S
FR-052	The system shall allow recording supplier payments against received POs, reducing	S

ID	Requirement	Priority
	outstanding balance.	
FR-053	Suppliers with existing purchase orders shall be deactivated rather than deleted.	M

4.7 Module 7: Purchase Orders

Records purchases from suppliers and feeds inventory automatically.

ID	Requirement	Priority
FR-060	The system shall support purchase order creation with supplier, expected delivery date, notes, and line items (ingredient, quantity, unit cost).	M
FR-061	Purchase orders shall follow the status flow: DRAFT → ORDERED → PARTIALLY_RECEIVED → RECEIVED, with CANCELLED allowed from DRAFT or ORDERED.	M
FR-062	Receiving a PO (fully or per-line partial quantities) shall create PURCHASE inventory transactions for the received quantities and update weighted average costs atomically.	M
FR-063	The system shall generate a sequential PO number per restaurant (e.g., PO-2026-00042).	M
FR-064	A PO in RECEIVED status shall be read-only; corrections require a new adjustment transaction or supplier return.	M
FR-065	The system shall show variance between ordered and received quantities per line.	S

4.8 Module 8: Recipe Management

Defines the bill of materials for menu items and drives automatic stock deduction.

ID	Requirement	Priority
FR-070	The system shall allow defining one recipe per menu item and per add-on, consisting of ingredient lines (ingredient, quantity, unit).	M
FR-071	The system shall compute and display recipe cost from ingredient weighted average costs, and show margin against the selling price.	S
FR-072	When an order is completed (see FR-085), the system shall create CONSUMPTION transactions deducting each recipe ingredient multiplied by ordered quantity, including add-ons, within the same database transaction as the status change.	M
FR-073	If deduction would drive any ingredient negative, the order completion shall fail with a descriptive error listing the insufficient ingredients (see BR-001).	M
FR-074	At order creation time, the system shall warn (non-blocking) when projected stock for any ingredient of the ordered items is insufficient.	S
FR-075	Recipe changes shall not affect historical consumption records.	M

4.9 Module 9: Order Management

The core selling workflow for dine-in, take-out, and delivery.

ID	Requirement	Priority
FR-080	The system shall support creating orders of type DINE_IN (requires table), TAKE_OUT, and DELIVERY (requires customer name, phone, and address).	M
FR-081	The system shall assign a per-restaurant, per-day sequential order number (e.g., #0042) plus a globally unique order reference.	M
FR-082	An order shall contain one or more order items with: menu item, quantity (≥1), unit price captured at time of sale, selected add-ons, and item notes (e.g., 'no onions').	M
FR-083	Orders shall follow the status flow: PENDING → PREPARING → READY → COMPLETED, with CANCELLED allowed before COMPLETED (per Section 3.2 permissions).	M
FR-084	Only PENDING orders may have items added, changed, or removed; PREPARING and later statuses lock the item list.	M
FR-085	COMPLETED shall be reached only when full payment has been recorded; completing	M

ID	Requirement	Priority
	an order triggers recipe deduction (FR-072). Completed orders are immutable (BR-002).	
FR-086	Cancelling an order shall require a reason; if any inventory was already consumed for it, compensating restore transactions shall be created (BR-003).	M
FR-087	The system shall support order-level discounts: Senior Citizen (20%, VAT-exempt per RA 9994), PWD (20%, VAT-exempt per RA 10754), and manual percentage/amount discounts (Manager/Owner approval required above 10%). Senior/PWD discounts require ID number capture.	M
FR-088	The kitchen queue screen shall list PENDING and PREPARING orders oldest-first, with item details and notes, and allow Kitchen Staff to advance status (PENDING→PREPARING→READY).	M
FR-089	The system shall timestamp every status change and display elapsed time per order in the kitchen queue.	S

4.10 Module 10: Table Management

Tracks the physical tables of the restaurant for dine-in service.

ID	Requirement	Priority
FR-090	The system shall support table CRUD with: name/number, seating capacity, and active flag.	M
FR-091	Tables shall have status AVAILABLE, OCCUPIED, RESERVED, or CLEANING; status changes are recorded with user and timestamp.	M
FR-092	Creating a dine-in order for a table shall set it to OCCUPIED; completing or cancelling all its open orders shall set it to CLEANING, then AVAILABLE when confirmed by staff.	M
FR-093	The system shall support transferring an open order to another AVAILABLE table.	M
FR-094	The system shall support merging two open dine-in orders onto one table into a single bill, preserving the item audit trail.	S
FR-095	The table view shall present a color-coded grid of all tables with status and running bill amount.	S

4.11 Module 11: Payments & Receipts

Records settlement of orders and issues receipts.

ID	Requirement	Priority
FR-100	The system shall accept payment methods: CASH, GCASH, CARD, and BANK_TRANSFER; non-cash methods require a reference number.	M
FR-101	The system shall support split payments: multiple payment records per order until the total is fully settled; overpayment is allowed only for CASH (change is computed and displayed).	M
FR-102	The system shall validate that the sum of captured payments equals or exceeds the order total before the order can be COMPLETED; each payment shall belong to exactly one order (BR-005).	M
FR-103	The system shall generate a printable receipt containing: restaurant name, address, TIN, VAT registration status, serial receipt number, date/time, cashier name, itemized lines, VATable sales, VAT-exempt sales, 12% VAT amount, discounts (with Senior/PWD ID where applicable), total, amount tendered, change, and the configured footer message.	M
FR-104	Receipt reprints shall be allowed and shall be marked 'REPRINT' and audit-logged.	M
FR-105	Refunds/voids shall be restricted to Owner/Manager, require a reason, create a negative payment record linked to the original, and appear in reports and the audit log.	M
FR-106	The receipt layout shall render correctly on 58mm and 80mm thermal printers via the browser print dialog.	S

4.12 Module 12: Expense Management

Captures operating expenses for accurate profit reporting.

ID	Requirement	Priority
FR-110	The system shall support expense CRUD with: date, category (Electricity, Water, Internet, Rent, Salary, Supplies, Maintenance, Gas/LPG, Transportation, Other), payee, amount, payment method, notes, and optional receipt image (Blob Storage, max 5 MB).	M
FR-111	Expenses above a configurable threshold (default ₱5,000) created by a Cashier shall require Owner/Manager approval before inclusion in reports.	S
FR-112	The system shall support recurring expense templates (e.g., monthly rent) that generate a draft expense on schedule for confirmation.	C
FR-113	Expenses for a closed business day shall not be editable (BR-008); corrections are entered as adjusting entries on an open day.	M

4.13 Module 13: Employee Management

Maintains staff records and attendance.

ID	Requirement	Priority
FR-120	The system shall support employee CRUD with: full name, nickname, phone, address, emergency contact, position, employment status (Active/Inactive), date hired, daily rate or monthly salary, and optional linked user account.	M
FR-121	Salary information shall be visible only to Owner and Manager.	M
FR-122	The system shall support attendance: clock-in and clock-out per employee per day, with edit capability for Owner/Manager (edits are audit-logged).	S
FR-123	The system shall produce a monthly attendance summary per employee (days present, total hours).	S

4.14 Module 14: Reports

Operational and financial reporting with export.

ID	Requirement	Priority
FR-130	Daily Sales Report: gross sales, discounts, net sales, VAT breakdown, payment method breakdown, order count, average order value, cancelled orders, refunds, per selected date.	M
FR-131	X-Reading (per cashier, mid-shift, non-resetting) and Z-Reading (end-of-day, per business day) reports.	S
FR-132	Monthly Sales Report: daily totals for the month with trend chart, compared to previous month.	M
FR-133	Profit & Loss Report: net sales – cost of goods (from consumption transactions) – expenses = profit, for any date range.	M
FR-134	Inventory Reports: current stock levels, inventory usage by ingredient over a range, spoilage report with cost.	M
FR-135	Product Mix Report: best and least selling items by quantity and revenue, by date range and category.	M
FR-136	Purchase Report: POs by supplier and range, with totals and outstanding balances.	S
FR-137	Tax Summary: VATable sales, VAT-exempt sales (Senior/PWD), and output VAT per range.	M
FR-138	All reports shall be filterable by date range and exportable to CSV and PDF; exports are generated asynchronously for ranges over 90 days and delivered via notification.	M
FR-139	Report figures shall be sourced from immutable records (orders, payments, transactions) so identical parameters always reproduce identical results.	M

4.15 Module 15: Daily Closing

End-of-day cash reconciliation and day locking.

ID	Requirement	Priority
FR-140	The system shall guide the Owner/Manager through daily closing: (1) verify no open orders remain (block or force-resolve), (2) system computes expected cash = cash sales – cash refunds – cash expenses + opening float, (3) user enters counted cash, (4) system computes and records variance.	M
FR-141	A variance beyond $\pm\$100$ shall require a written explanation before closing can complete.	S
FR-142	Completing closing shall lock the business day: no orders, payments, or expenses may be created or modified for that day (BR-008).	M
FR-143	The system shall generate an End-of-Day report (Z-Reading + expense summary + variance) stored immutably and exportable to PDF.	M
FR-144	Only the Owner may reopen a closed day; reopening requires a reason and is prominently audit-logged; the day must be re-closed the same calendar week.	M
FR-145	The system shall record the opening cash float at the start of each business day (defaults to previous closing float).	S

4.16 Module 16: Notifications

Keeps staff informed of operationally significant events.

ID	Requirement	Priority
FR-150	The system shall generate in-app notifications for: low stock (FR-045), new order placed (to Kitchen), order cancelled (to Kitchen and Manager), large discount applied, daily closing reminder (configurable time, default 21:00), and PO received.	M
FR-151	Notifications shall be delivered to roles relevant to the event (per Section 3.2) and appear within 5 seconds for order events.	M
FR-152	Users shall be able to view a notification center with unread count, mark items read, and mark all read.	M
FR-153	Notifications older than 90 days shall be purged automatically.	S

4.17 Module 17: Audit Logs

Immutable, append-only trail of significant actions for accountability.

ID	Requirement	Priority
FR-160	The system shall record an audit entry for: login success/failure, logout, all create/update/delete operations on business entities, status transitions (orders, POs, tables), payments, refunds, reprints, discounts above 10%, day close/reopen, exports, and permission-denied attempts.	M
FR-161	Each entry shall record: timestamp (UTC), user, role, restaurant, action, entity type and id, before/after snapshot (JSON) for updates, IP address, and user agent.	M
FR-162	Audit logs shall be append-only; no API shall exist to modify or delete entries; retention is a minimum of 1 year online.	M
FR-163	Owners (full) and Managers (read) shall be able to search logs by user, entity, action, and date range.	M

5. User Stories

User stories are grouped by epic and expressed with acceptance criteria in Given/When/Then form. Story points use a Fibonacci scale (1, 2, 3, 5, 8). Each story traces to the functional requirements it realizes; a story is Done only when its acceptance criteria pass automated tests and code review (see Section 16).

Epic E1 — Authentication & Accounts

US-001 — Log in

User Story	As a Cashier, I want to log in with my email and password, so that I can start taking orders at the beginning of my shift.
Acceptance Criteria	Given valid credentials, when I submit the login form, then I receive access and refresh tokens and land on the dashboard within 3 seconds. Given an invalid password, when I submit, then I see 'Invalid email or password' without revealing which field was wrong. Given 5 consecutive failures, when I try again, then I am blocked for 15 minutes and the lockout is audit-logged.
Priority / Points	Must / 3 pts
Traces To	FR-001, FR-002, FR-003, FR-009

US-002 — Reset forgotten password

User Story	As a Owner, I want to reset my password by email, so that I can regain access without contacting support.
Acceptance Criteria	Given a registered email, when I request a reset, then I receive a single-use link valid for 1 hour. Given an unregistered email, when I request a reset, then the UI shows the same confirmation message (no account enumeration). Given a used or expired link, when I open it, then I see an error and can request a new link.
Priority / Points	Must / 3 pts
Traces To	FR-005

US-003 — Create staff accounts

User Story	As a Owner, I want to create accounts for my staff with a role, so that each employee has their own login and appropriate access.
Acceptance Criteria	Given I am an Owner, when I create a user with role Cashier, then they can log in and see only Cashier features. Given a Manager, when they try to create another Manager or Owner, then the API rejects with 403. Given a new staff account, when the user first logs in, then they are forced to set a new password.
Priority / Points	Must / 5 pts
Traces To	FR-012, FR-013, FR-014

US-004 — Deactivate a departed employee's account

User Story	As a Owner, I want to deactivate a user who left the business, so that former staff can no longer access our data.
Acceptance Criteria	Given an active user, when I deactivate them, then their current session is invalidated at next token refresh (≤15 min) and login is refused immediately. Given the last active Owner account, when I try to deactivate it, then the system refuses

	with an explanatory message. Given a deactivated user, when I view past orders they handled, then their name still appears.
Priority / Points	Must / 3 pts
Traces To	FR-015, FR-016

Epic E2 — Menu & Recipes

US-010 — Manage menu items

User Story	As a Manager, I want to add and edit menu items with photos and prices, so that customers are always ordering from an accurate menu.
Acceptance Criteria	Given a new item with name, category, and price, when I save, then it appears on the order screen immediately. Given an image over 5 MB or a non-image file, when I upload, then the system rejects it with a clear message. Given a price change, when I save, then the old price is kept in price history and past orders are unaffected.
Priority / Points	Must / 5 pts
Traces To	FR-031, FR-032, FR-036

US-011 — Mark an item sold out

User Story	As a Cashier, I want to see sold-out items disabled, so that I never sell something the kitchen cannot make.
Acceptance Criteria	Given the Manager toggled 'Tapsilog' unavailable, when I open the order screen, then Tapsilog is visibly disabled and cannot be added. Given the item is re-enabled, when I refresh, then it is orderable again.
Priority / Points	Must / 2 pts
Traces To	FR-034, FR-037

US-012 — Define a recipe

User Story	As a Owner, I want to define ingredient quantities per menu item, so that stock is deducted automatically and I know my food cost.
Acceptance Criteria	Given Tapsilog = 200 g beef + 1 egg + 250 g rice, when I save the recipe, then the recipe cost is computed from weighted average costs and margin is shown. Given a completed order of 2 Tapsilog, when completion succeeds, then 400 g beef, 2 eggs, and 500 g rice CONSUMPTION transactions exist referencing the order.
Priority / Points	Must / 5 pts
Traces To	FR-070, FR-071, FR-072

Epic E3 — Inventory, Suppliers & Purchasing

US-020 — Track ingredient stock

User Story	As a Manager, I want to see current stock and a full movement history per ingredient, so that I can trust the numbers without a physical count every day.
Acceptance Criteria	Given any ingredient, when I open its stock card, then I see a running balance where each row shows type, signed quantity, resulting balance, reference, user, and timestamp. Given any filter (date range, type), when applied, then the visible rows still reconcile to the displayed balances.
Priority / Points	Must / 5 pts

Traces To	FR-041, FR-043, FR-046
-----------	------------------------

US-021 — Record spoilage

User Story	As a Manager, I want to record spoiled ingredients with a reason, so that wastage is visible in reports instead of silently distorting stock.
Acceptance Criteria	Given 2 kg of beef spoiled, when I record SPOILAGE with reason, then stock decreases by 2 kg and the spoilage report shows the cost. Given a spoilage quantity larger than current stock, when I submit, then the system rejects it (stock cannot go negative).
Priority / Points	Must / 3 pts
Traces To	FR-042, FR-044, FR-134

US-022 — Get low-stock warnings

User Story	As a Owner, I want to be notified when ingredients hit minimum stock, so that I can reorder before we run out mid-service.
Acceptance Criteria	Given rice minimum stock is 10 kg, when a deduction brings it to 10 kg or below, then a low-stock notification is created once and appears on the dashboard. Given stock is replenished above 10 kg and later drops again, then a new notification is created.
Priority / Points	Must / 3 pts
Traces To	FR-045, FR-150

US-023 — Receive a purchase order

User Story	As a Manager, I want to receive deliveries against a PO, including partial deliveries, so that stock and costs update without manual entry.
Acceptance Criteria	Given an ORDERED PO for 50 kg rice, when I receive 30 kg, then the PO becomes PARTIALLY_RECEIVED, stock increases by 30 kg, and weighted average cost is recalculated. Given the remaining 20 kg arrives, when I receive it, then the PO becomes RECEIVED and is read-only thereafter. Given a RECEIVED PO, when anyone attempts to edit it, then the API rejects with 409.
Priority / Points	Must / 8 pts
Traces To	FR-061, FR-062, FR-064, FR-047

Epic E4 — Orders, Tables & Kitchen

US-030 — Take a dine-in order

User Story	As a Cashier, I want to create an order for a table quickly, so that customers are served promptly and the kitchen is informed instantly.
Acceptance Criteria	Given table T3 is AVAILABLE, when I create a dine-in order with 2 items, then T3 becomes OCCUPIED and the order appears in the kitchen queue within 5 seconds. Given the customer adds an item while the order is PENDING, when I edit the order, then the change succeeds; once PREPARING, editing is blocked. Given an item note 'no onions', when the kitchen views the order, then the note is visible on the item line.
Priority / Points	Must / 8 pts
Traces To	FR-080, FR-082, FR-084, FR-088, FR-092, FR-151

US-031 — Work the kitchen queue

User Story	As a Kitchen Staff, I want to see pending orders oldest-first and advance their status, so that the dining area always knows what is ready.
Acceptance Criteria	Given three orders placed at different times, when I open the queue, then they are sorted oldest-first with elapsed time displayed. Given a PENDING order, when I tap Start, then it becomes PREPARING; when I tap Ready, then it becomes READY and the cashier is notified. Given my role is Kitchen, when I try to modify order items or take payment, then the actions are unavailable and the API refuses.
Priority / Points	Must / 5 pts
Traces To	FR-083, FR-088, FR-089

US-032 — Apply a Senior Citizen discount

User Story	As a Cashier, I want to apply the mandatory 20% senior discount with ID capture, so that we comply with RA 9994 and the receipt reflects the exemption.
Acceptance Criteria	Given a senior with ID, when I apply the Senior discount and enter the ID number, then 20% is deducted and the sale is marked VAT-exempt. Given the receipt, when printed, then it shows the discount, the Senior ID number, and VAT-exempt sales on the tax breakdown. Given a manual discount above 10% instead, when I attempt it as Cashier, then Manager/Owner approval is required.
Priority / Points	Must / 5 pts
Traces To	FR-087, FR-103, FR-137

US-033 — Cancel an order

User Story	As a Manager, I want to cancel a wrong order with a reason, so that stock and sales figures stay accurate.
Acceptance Criteria	Given a PREPARING order whose stock was not yet consumed, when I cancel with a reason, then the order is CANCELLED and no inventory changes exist. Given inventory had been consumed, when I cancel, then compensating restore transactions are created referencing the order. Given a COMPLETED order, when I attempt to cancel, then the system refuses (immutable) and points me to the refund flow.
Priority / Points	Must / 5 pts
Traces To	FR-086, BR-002, BR-003

US-034 — Transfer and merge tables

User Story	As a Cashier, I want to move a party to another table or merge two bills, so that table status and bills stay correct when customers move.
Acceptance Criteria	Given an open order on T2 and T5 AVAILABLE, when I transfer to T5, then T5 is OCCUPIED, T2 goes to CLEANING, and the order history records the transfer. Given open orders on T2 and T3 whose parties merged, when I merge onto T3, then a single bill contains all items with their original timestamps.
Priority / Points	Should / 5 pts
Traces To	FR-093, FR-094

Epic E5 — Payments & Receipts

US-040 — Accept a cash payment with change

User Story	As a Cashier, I want to enter the amount tendered and see the change, so that checkout is fast and error-free.
Acceptance Criteria	Given a ₱285.00 order and ₱500 tendered, when I record the cash payment, then change ₱215.00 is displayed and printed on the receipt. Given insufficient tendered amount, when I try to complete the order, then completion is blocked until the balance is settled.
Priority / Points	Must / 3 pts
Traces To	FR-100, FR-101, FR-102, FR-103

US-041 — Split a bill

User Story	As a Cashier, I want to accept part cash and part GCash for one order, so that groups can pay the way they want.
Acceptance Criteria	Given a ₱1,000 order, when I record ₱600 GCash (with reference number) and ₱400 cash, then the order can be completed and the receipt lists both payments. Given a GCash payment without a reference number, when I submit, then validation fails.
Priority / Points	Must / 5 pts
Traces To	FR-100, FR-101, FR-102

US-042 — Refund a payment

User Story	As a Owner, I want to refund a completed order with a reason, so that mistakes are corrected transparently.
Acceptance Criteria	Given a completed order, when I issue a refund with a reason, then a negative payment linked to the original is created and appears in the daily sales report and audit log. Given my role is Cashier, when I attempt a refund, then the action is refused.
Priority / Points	Must / 5 pts
Traces To	FR-105, FR-130, FR-160

Epic E6 — Expenses, Employees & Closing

US-050 — Record a daily expense

User Story	As a Cashier, I want to record small purchases like LPG with a photo of the receipt, so that daily profit reflects true costs.
Acceptance Criteria	Given an expense with category, amount, and photo, when I save, then it appears in today's expenses pending any required approval. Given an amount above the approval threshold, when saved by a Cashier, then it is excluded from reports until approved by Owner/Manager.
Priority / Points	Must / 3 pts
Traces To	FR-110, FR-111

US-051 — Track attendance

User Story	As a Manager, I want to record staff clock-in/clock-out, so that attendance summaries are available for payroll.
Acceptance Criteria	Given an employee, when they clock in and out, then the day's hours are computed and appear in the monthly summary. Given a missed punch, when I correct it, then the edit is audit-logged with before/after values.
Priority / Points	Should / 3 pts
Traces To	FR-122, FR-123

US-052 — Close the day

User Story	As a Owner, I want to reconcile cash and lock the business day, so that records cannot be quietly altered after closing.
Acceptance Criteria	Given open orders exist, when I start closing, then I am blocked until they are completed or cancelled. Given expected cash ₱12,540 and counted ₱12,440, when I enter the count, then variance -₱100 is recorded; beyond ±₱100 an explanation is mandatory. Given the day is closed, when anyone tries to add or edit that day's orders, payments, or expenses, then the API refuses with 423 (Locked). Given a closed day, when I (Owner) reopen it with a reason, then the reopening is audit-logged and the day must be re-closed.
Priority / Points	Must / 8 pts
Traces To	FR-140, FR-141, FR-142, FR-144, BR-008

Epic E7 — Reporting & Insight

US-060 — See today at a glance

User Story	As a Owner, I want a dashboard of sales, orders, top sellers, and low stock, so that I can check business health from anywhere in under a minute.
Acceptance Criteria	Given sales exist today, when I open the dashboard, then today's gross/net sales, order counts by status, top 5 items, and low-stock list are correct versus the underlying reports. Given I am a Cashier, when I open the dashboard, then financial widgets are hidden.
Priority / Points	Must / 5 pts
Traces To	FR-020..FR-026

US-061 — Understand profit

User Story	As a Owner, I want a profit & loss report for any date range, so that I know whether the business actually makes money.
Acceptance Criteria	Given a date range, when I run P&L, then profit = net sales – COGS (from consumption transactions) – approved expenses, and each figure drills down to its source records. Given identical parameters run twice, then results are identical (immutable sources).
Priority / Points	Must / 5 pts
Traces To	FR-133, FR-139

US-062 — Export reports

User Story	As a Owner, I want to export any report to CSV and PDF, so that I can share figures with my accountant.
Acceptance Criteria	Given any report, when I export CSV, then columns match the on-screen table including totals. Given a range over 90 days, when I export, then generation runs in the background and a notification delivers the download link.
Priority / Points	Must / 3 pts
Traces To	FR-138

6. Use Case Specifications

The following use cases elaborate the highest-risk workflows end to end. Primary actors are tenant users; the system boundary is the TAPSI web application and API.

UC-01: Log In

Primary Actor(s)	Owner, Manager, Cashier, Kitchen Staff
Preconditions	User has an active account. User is not locked out.
Main Success Scenario	1. User opens the login page and enters email and password. 2. System validates credentials and account status. 3. System issues access (15 min) and refresh (7 days) tokens. 4. System records the successful login (audit) and redirects to the role-appropriate landing screen.
Extensions / Alternate Flows	2a. Invalid credentials: system shows a generic error; failure count increments. 2b. Fifth consecutive failure: account locks for 15 minutes; lockout audit-logged. 2c. Deactivated account: login refused with 'Account disabled — contact your Owner.'
Postconditions	User is authenticated; session activity is traceable.
Traces To	FR-001..FR-004, FR-009

UC-02: Create Dine-In Order

Primary Actor(s)	Cashier
Preconditions	Cashier is authenticated. At least one AVAILABLE table exists. Menu has available items.
Main Success Scenario	1. Cashier selects table and order type DINE_IN. 2. Cashier adds menu items (with quantities, add-ons, item notes); system shows a running total and any non-blocking low-stock warnings. 3. Cashier confirms; system assigns the daily order number, sets status PENDING, and sets the table to OCCUPIED. 4. System notifies the kitchen queue within 5 seconds.
Extensions / Alternate Flows	2a. Item unavailable: item is disabled and cannot be added. 3a. Table became OCCUPIED meanwhile: system rejects and prompts to pick another table. 4a. Customer changes mind while PENDING: cashier edits items; once PREPARING, item list is locked.
Postconditions	Order exists with status PENDING; kitchen sees it; table OCCUPIED.
Traces To	FR-080..FR-084, FR-088, FR-092

UC-03: Settle Payment and Complete Order

Primary Actor(s)	Cashier
Preconditions	Order is READY (or PENDING/PREPARING for take-out). Order total is computed with any discounts.
Main Success Scenario	1. Cashier opens the order and selects Pay. 2. Cashier records one or more payments (method, amount, reference for non-cash) until the total is settled; system computes change for cash. 3. Cashier confirms completion. 4. In a single database transaction the system: verifies payments cover the total, creates recipe CONSUMPTION deductions (order items + add-ons), marks the order COMPLETED, and increments the receipt serial. 5. System renders the BIR-compliant receipt for printing.
Extensions / Alternate Flows	4a. Any ingredient would go negative: transaction rolls back; system lists insufficient ingredients; staff resolves stock (adjustment/receiving) and retries. 2a. Senior/PWD discount: cashier captures ID number; 20% discount and VAT exemption

	are applied before payment. 5a. Reprint requested later: allowed, watermarked REPRINT, audit-logged.
Postconditions	Order COMPLETED and immutable; inventory reflects consumption; receipt issued.
Traces To	FR-072, FR-073, FR-085, FR-087, FR-100..FR-104, BR-001, BR-002

UC-04: Cancel Order and Restore Inventory

Primary Actor(s)	Manager (or Owner)
Preconditions	Order exists and is not COMPLETED.
Main Success Scenario	1. Manager opens the order and selects Cancel. 2. System requires a cancellation reason. 3. System sets status CANCELLED; if consumption transactions exist for the order, system creates compensating restore transactions. 4. System notifies kitchen and records the cancellation in the audit log.
Extensions / Alternate Flows	1a. Cashier initiates: request goes to Manager/Owner for approval. 3a. Order already COMPLETED: refuse; direct user to the refund flow (FR-105).
Postconditions	Order CANCELLED; stock restored to pre-order levels; cancellation reported in daily sales.
Traces To	FR-086, FR-105, BR-002, BR-003

UC-05: Receive Purchase Order (Partial)

Primary Actor(s)	Manager
Preconditions	PO exists in ORDERED status.
Main Success Scenario	1. Manager opens the PO and selects Receive. 2. Manager enters received quantity per line (may differ from ordered). 3. System validates quantities (> 0, $\leq$ outstanding per line). 4. In a single transaction: PURCHASE inventory transactions are created, weighted average costs recalculated, and PO status set to PARTIALLY_RECEIVED or RECEIVED. 5. System clears any low-stock notifications for replenished ingredients.
Extensions / Alternate Flows	3a. Quantity exceeds outstanding: line rejected with message. 4a. Concurrent receive of the same PO: second transaction fails on optimistic lock and is retried against fresh state.
Postconditions	Stock and costs updated; PO reflects cumulative received quantities; RECEIVED POs are read-only.
Traces To	FR-060..FR-065, FR-047

UC-06: Perform Daily Closing

Primary Actor(s)	Owner or Manager
Preconditions	Business day has recorded activity. User holds closing permission.
Main Success Scenario	1. User starts Daily Closing for the current business day. 2. System verifies no open orders remain (lists them if any). 3. System presents expected cash: opening float + cash sales – cash refunds – cash expenses. 4. User enters counted cash; system computes variance. 5. If variance exceeds $\pm\\$100$, system requires an explanation. 6. User confirms; system locks the day, generates the End-of-Day report, and stores it immutably.
Extensions / Alternate Flows	2a. Open orders exist: user completes or cancels each before proceeding. 6a. Reopening later: Owner only, with reason; prominently audit-logged; day must be re-closed.
Postconditions	Day locked against modification; EOD report available as PDF.
Traces To	FR-140..FR-145, BR-008

UC-07: Add Menu Item with Recipe

Primary Actor(s)	Manager
Preconditions	Ingredients used by the recipe already exist.
Main Success Scenario	1. Manager creates the menu item (name, category, price, prep time, image). 2. Manager opens the Recipe tab and adds ingredient lines with quantities. 3. System computes recipe cost and gross margin against the selling price. 4. Manager saves; item becomes orderable (if marked available).
Extensions / Alternate Flows	1a. Image invalid (>5 MB or wrong type): upload rejected. 2a. A needed ingredient does not exist: Manager creates it inline, then continues. 3a. Margin is negative: system warns but does not block.
Postconditions	Item sellable; future completions deduct stock; cost basis visible.
Traces To	FR-031, FR-032, FR-070, FR-071

UC-08: Investigate a Stock Discrepancy

Primary Actor(s)	Owner
Preconditions	Physical count differs from system stock.
Main Success Scenario	1. Owner opens the ingredient's stock card and reviews the transaction ledger. 2. Owner cross-references order consumption, PO receipts, spoilage, and adjustments via each transaction's reference link. 3. Owner records an ADJUSTMENT transaction with the counted difference and a mandatory reason. 4. System updates the balance and the adjustment appears in inventory reports and the audit log.
Extensions / Alternate Flows	3a. Adjustment would make stock negative: rejected; Owner re-checks the count.
Postconditions	Ledger reconciles with physical reality; the correction is fully traceable.
Traces To	FR-042, FR-043, FR-044, FR-046, FR-163

7. Non-Functional Requirements

7.1 Performance

ID	Requirement	Priority
NFR-001	95th percentile API response time shall be < 300 ms and 99th percentile < 800 ms for standard CRUD endpoints under nominal load (measured at the Azure Front Door).	M
NFR-002	Report endpoints may take up to 3 s synchronously; anything longer shall run asynchronously with progress notification (see FR-138).	M
NFR-003	The system shall support 100 concurrent active users per application instance with the stated response times; capacity shall scale horizontally (NFR-010).	M
NFR-004	The order screen (menu grid) shall become interactive within 2 s on a mid-range Android tablet over 4G.	S
NFR-005	Database queries backing list screens shall be paginated (default 25, max 100 rows) and covered by indexes; no endpoint shall trigger unbounded table scans on tenant data.	M

7.2 Availability & Reliability

ID	Requirement	Priority
NFR-010	The service shall achieve 99.9% monthly availability (≤ 43.8 minutes downtime/month), excluding announced maintenance windows (≤ 4 h/month, off-peak 02:00–06:00 PHT).	M

ID	Requirement	Priority
NFR-011	The application tier shall run at least 2 instances across 2 Availability Zones behind Azure Front Door; Azure Database for PostgreSQL shall run zone-redundant.	M
NFR-012	All money- and stock-affecting operations shall be atomic database transactions; a failure shall never leave partial state (e.g., payment recorded without order completion).	M
NFR-013	The system shall degrade gracefully: if the notification or reporting subsystem is down, order taking and payment shall continue to function.	S
NFR-014	Idempotency: order creation and payment endpoints shall accept a client-generated idempotency key so retried requests do not create duplicates.	M

7.3 Security

ID	Requirement	Priority
NFR-020	Every endpoint shall enforce authentication and role-based authorization server-side; object-level checks shall verify tenant ownership (no IDOR).	M
NFR-021	Passwords shall be hashed with Argon2id (or Django's default PBKDF2 with $\geq 600k$ iterations); no credential shall ever be logged or stored in plaintext.	M
NFR-022	All traffic shall use HTTPS (TLS 1.2+); HSTS enabled; cookies (if any) Secure/HttpOnly/SameSite.	M
NFR-023	The system shall protect against OWASP Top 10: parameterized queries via the ORM (SQL injection), output encoding and CSP (XSS), CSRF protection on any cookie-based flows, and strict input validation via DRF serializers.	M
NFR-024	Rate limiting: authentication endpoints max 10 requests/min/IP; general API max 120 requests/min/user.	M
NFR-025	Secrets (DB credentials, JWT signing key, Azure Communication Services Email keys) shall be stored in Azure Key Vault, never in code or images; Azure RBAC / managed identities follow least privilege.	M
NFR-026	Personal data (employee records, customer delivery details) shall be handled per the Data Privacy Act of 2012: collected minimally, access-controlled, and erasable on verified request.	M
NFR-027	Blob Storage containers shall be private; images served via Azure Front Door signed URLs or restricted network access; uploads virus-size-type validated server-side.	M
NFR-028	Dependency and container images shall be scanned in CI; critical CVEs block deployment.	S

7.4 Scalability & Capacity

ID	Requirement	Priority
NFR-030	The application shall be stateless (no local session or file state) so instances can be added/removed by an App Service scale-out based on CPU ($> 70\%$) and request latency.	M
NFR-031	Design capacity for v1.0: 500 tenant restaurants, 20 users each, 2,000 orders/day/tenant peak, 3-year retention of transactional data (~50 GB).	S
NFR-032	Static assets and images shall be served from Azure Front Door/CDN, not application instances.	M

7.5 Usability & Accessibility

ID	Requirement	Priority
NFR-040	A new cashier shall be able to take an order and settle payment after ≤ 15 minutes of instruction, without a manual.	S
NFR-041	The order and kitchen screens shall be operable on a touch tablet: touch targets $\geq 44 \times 44$ px, no hover-dependent actions.	M
NFR-042	UI text shall be in English with Filipino-friendly terminology; all money displayed with the ₱ symbol and two decimals.	M

ID	Requirement	Priority
NFR-043	Core flows shall meet WCAG 2.1 AA contrast and keyboard navigation.	C
NFR-044	Destructive actions (cancel order, delete, refund, reopen day) shall require an explicit confirmation naming the object affected.	M

7.6 Maintainability & Portability

ID	Requirement	Priority
NFR-050	Backend code shall maintain $\geq 80\%$ line coverage on business-logic modules (orders, payments, inventory, closing); CI blocks merges below threshold.	M
NFR-051	All schema changes shall be applied via Django migrations; no manual DDL in any environment.	M
NFR-052	The system shall run identically in local Docker Compose, staging, and production (12-factor configuration via environment).	M
NFR-053	API changes shall be backward compatible within /api/v1; breaking changes require /api/v2.	S
NFR-054	Logs shall be structured (JSON) with request IDs enabling end-to-end tracing of a single request.	M

8. Business Rules

Business rules are invariants that hold regardless of which feature or interface triggers the operation. They are enforced in the service layer and, where possible, by database constraints.

ID	Rule	Enforcement
BR-001	Ingredient stock can never be negative.	DB CHECK on running balance + service-layer validation with row-level locking (SELECT ... FOR UPDATE) during deduction.
BR-002	Completed orders are immutable: no edits to items, amounts, or discounts.	Status guard in service layer; UPDATE forbidden by model logic; corrections via refund (FR-105).
BR-003	Cancelling an order restores any inventory already consumed for it.	Compensating transactions created atomically with the status change.
BR-004	Recipes deduct inventory automatically upon order completion — never at order creation.	Deduction executes inside the completion DB transaction (UC-03).
BR-005	Every payment belongs to exactly one order; a payment can never exceed what remains unpaid except cash (change).	FK NOT NULL + service validation.
BR-006	Only Owners may delete or deactivate users; the last active Owner cannot be removed.	Permission class + service check (FR-015, FR-016).
BR-007	Deleted (soft-deleted) menu items, users, and suppliers remain visible in historical records and reports.	Soft-delete flag; historical rows denormalize name and price at time of sale.
BR-008	A closed business day is locked: no creation or modification of that day's orders, payments, or expenses; only an Owner may reopen with a reason.	Day-lock check in service layer returns HTTP 423; reopen audit-logged.
BR-009	Senior Citizen and PWD discounts are 20% and VAT-exempt, and require capture of the ID number.	Discount engine (FR-087); receipt shows exemption (FR-103).
BR-010	Prices, costs, and totals are stored to 2 decimal places in PHP; rounding is half-up at line level, then summed.	NUMERIC(12,2) columns; Decimal arithmetic only.
BR-011	Order totals: subtotal = $\sum(\text{line qty} \times \text{unit price} + \text{add-ons})$; total = subtotal – discounts; VAT (12%) is computed on VATable sales only.	Single pricing service used by API, receipts, and reports.
BR-012	Inventory transactions are immutable; corrections are	No UPDATE/DELETE endpoints; DB

ID	Rule	Enforcement
	compensating entries, never edits or deletions.	permissions deny UPDATE on the table.
BR-013	All figures in reports derive from immutable source records so any report is reproducible.	Reports read from orders/payments/transactions; no cached mutable aggregates as source of truth.
BR-014	A business day runs 00:00–23:59 Asia/Manila; every order, payment, and expense belongs to exactly one business day (its creation date), which drives day locking.	business_date column stamped at creation.

9. Database Design

9.1 Design Principles

- Multi-tenancy: every tenant-owned table carries restaurant_id (FK, NOT NULL, indexed); all application queries are tenant-scoped by a base manager.
- Money: NUMERIC(12,2); quantities: NUMERIC(12,3). No floats.
- Soft deletes (is_deleted, deleted_at) on menu_item, category, user, supplier, employee; historical documents denormalize names/prices at time of transaction.
- Immutable ledgers: inventory_transaction, payment, audit_log are append-only.
- All tables: id BIGSERIAL PK, created_at/updated_at TIMESTAMPTZ (UTC).
- Standard composite indexes: (restaurant_id, business_date) on orders, payments, expenses; (restaurant_id, ingredient_id, created_at) on inventory_transaction.

9.2 Entity Relationship Overview

```

restaurant 1--* user
restaurant 1--* category 1--* menu_item
menu_item 1--1 recipe 1--* recipe_ingredient *--1 ingredient
ingredient 1--* inventory_transaction
supplier 1--* purchase_order
purchase_order 1--* purchase_order_item *--1 ingredient
restaurant 1--* dining_table
dining_table 1--* order (dine-in)
order 1--* order_item (*--1 menu_item)
order_item 1--* order_item_addon
order 1--* payment
restaurant 1--* expense
restaurant 1--* employee 1--* attendance
restaurant 1--* daily_closing
restaurant 1--* notification
restaurant 1--* audit_log

```

Cardinality: 1--* is one-to-many; 1--1 one-to-one; *--1 many-to-one.

9.3 Data Dictionary

Common columns (id, restaurant_id, created_at, updated_at) are listed once here and omitted from the tables below for brevity: id BIGSERIAL PK; restaurant_id BIGINT FK→restaurant NOT NULL (except restaurant itself); created_at/updated_at TIMESTAMPTZ NOT NULL.

restaurant

Tenant root entity.

Column	Type	Constraints	Description
name	VARCHAR(120)	NOT NULL	Business name
address	TEXT		Business address printed on receipts
phone	VARCHAR(20)		Contact number

Column	Type	Constraints	Description
tin	VARCHAR(20)		BIR Tax Identification Number
is_vat_registered	BOOLEAN	DEFAULT true	Drives receipt tax block
logo_key	VARCHAR(255)		Blob Storage object key
receipt_footer	VARCHAR(255)		e.g., 'Salamat po! Balik kayo!'
closing_reminder_at	TIME	DEFAULT 21:00	Daily closing reminder time
is_active	BOOLEAN	DEFAULT true	Subscription status

user

Staff login accounts (custom Django user).

Column	Type	Constraints	Description
email	VARCHAR(254)	UNIQUE NOT NULL	Login identifier
password	VARCHAR(128)	NOT NULL	Argon2id/PBKDF2 hash
full_name	VARCHAR(120)	NOT NULL	
phone	VARCHAR(20)		
role	VARCHAR(16)	CHECK IN (OWNER, MANAGER, CASHIER, KITCHEN)	Single role per user
must_change_password	BOOLEAN	DEFAULT true	Set for new staff accounts
failed_login_count	SMALLINT	DEFAULT 0	Lockout counter
locked_until	TIMESTAMP TZ	NULL	Lockout expiry
is_active	BOOLEAN	DEFAULT true	Deactivation flag
is_deleted	BOOLEAN	DEFAULT false	Soft delete

category / menu_item

Sellable catalog.

Column	Type	Constraints	Description
category.name	VARCHAR(80)	UNIQUE(restaurant_id, name)	e.g., Silog Meals
category.display_order	SMALLINT	DEFAULT 0	Sort on order screen
menu_item.category_id	BIGINT	FK→category NOT NULL	
menu_item.name	VARCHAR(120)	NOT NULL	e.g., Tapsilog
menu_item.description	TEXT		
menu_item.price	NUMERIC(12,2)	CHECK ≥ 0	Current selling price
menu_item.image_key	VARCHAR(255)		Blob Storage key; thumbnail derived
menu_item.prep_minutes	SMALLINT	DEFAULT 10	
menu_item.is_available	BOOLEAN	DEFAULT true	Sold-out toggle
menu_item.is_deleted	BOOLEAN	DEFAULT false	Soft delete (BR-007)

price_history / addon

Price audit and add-on catalog.

Column	Type	Constraints	Description
price_history.menu_item_id	BIGINT	FK NOT NULL	
price_history.old_price / new_price	NUMERIC(12,2)	NOT NULL	FR-036
price_history.changed_by	BIGINT	FK→user	
addon.name	VARCHAR(80)	NOT NULL	e.g., Extra Rice
addon.price	NUMERIC(12,2)	CHECK ≥ 0	
menu_item_addon.menu_item_id / addon_id	BIGINT	composite UNIQUE	M:N attachment

ingredient

Stock-keeping units.

Column	Type	Constraints	Description
name	VARCHAR(120)	UNIQUE(restaurant_id, name)	e.g., Beef Tapa
unit	VARCHAR(8)	CHECK IN (g, kg, ml, L, pc, pack)	Unit of measure
current_stock	NUMERIC(12,3)	CHECK ≥ 0	Maintained from ledger (BR-001)
min_stock	NUMERIC(12,3)	DEFAULT 0	Low-stock threshold
avg_cost	NUMERIC(12,4)	DEFAULT 0	Weighted average unit cost
default_supplier_id	BIGINT	FK→supplier NULL	

inventory_transaction

Append-only stock ledger (BR-012).

Column	Type	Constraints	Description
ingredient_id	BIGINT	FK NOT NULL	
type	VARCHAR(12)	CHECK IN (PURCHASE, CONSUMPTION, ADJUSTMENT, SPOILAGE, RETURN)	FR-042
quantity	NUMERIC(12,3)	NOT NULL, signed	+in / -out
unit_cost	NUMERIC(12,4)	NULL	For PURCHASE lines
balance_after	NUMERIC(12,3)	NOT NULL CHECK ≥ 0	Running balance snapshot
reason	VARCHAR(255)	required for ADJUSTMENT/SPOILAGE	
ref_type / ref_id	VARCHAR(16) / BIGINT	NULL	Link to order / PO
created_by	BIGINT	FK→user NOT NULL	

supplier / purchase_order / purchase_order_item

Procurement.

Column	Type	Constraints	Description
supplier.name	VARCHAR(120)	NOT NULL	
supplier.payment_terms	VARCHAR(8)	CHECK IN (COD, 7D, 15D, 30D)	
po.po_number	VARCHAR(20)	UNIQUE(restaurant_id, po_number)	PO-2026-00042 (FR-063)
po.supplier_id	BIGINT	FK NOT NULL	
po.status	VARCHAR(20)	CHECK per FR-061	DRAFT...RECEIVED/CANCELLED
po.expected_date	DATE		
po.total	NUMERIC(12,2)	derived	Σ line qty × cost
po_item.ingredient_id	BIGINT	FK NOT NULL	
po_item.qty_ordered / qty_received	NUMERIC(12,3)	received ≤ ordered per receiving rules	Partial receiving (FR-062)
po_item.unit_cost	NUMERIC(12,4)	NOT NULL	

recipe / recipe_ingredient

Bill of materials.

Column	Type	Constraints	Description
recipe.menu_item_id	BIGINT	FK UNIQUE NOT NULL	One recipe per item (FR-070)
recipe_ingredient.recipe_id	BIGINT	FK NOT NULL	
recipe_ingredient.ingredient_id	BIGINT	FK NOT NULL	
recipe_ingredient.quantity	NUMERIC(12,3)	CHECK > 0	Per one serving

dining_table

Physical tables.

Column	Type	Constraints	Description
name	VARCHAR(40)	UNIQUE(restaurant_id, name)	e.g., T3
capacity	SMALLINT	DEFAULT 4	
status	VARCHAR(12)	CHECK IN (AVAILABLE, OCCUPIED, RESERVED, CLEANING)	FR-091
is_active	BOOLEAN	DEFAULT true	

order / order_item / order_item_addon

Sales documents.

Column	Type	Constraints	Description
order.order_no	SMALLINT	UNIQUE(restaurant_id, business_date, order_no)	Daily sequence (FR-081)
order.reference	UUID	UNIQUE	Global reference
order.type	VARCHAR(10)	CHECK IN (DINE_IN, TAKE_OUT, DELIVERY)	
order.table_id	BIGINT	FK NULL; NOT NULL when DINE_IN	
order.status	VARCHAR(10)	CHECK per FR-083	
order.business_date	DATE	NOT NULL	Drives day locking (BR-014)
order.subtotal / discount_amount / total	NUMERIC(12,2)	NOT NULL	Per BR-011
order.discount_type / discount_ref	VARCHAR(12) / VARCHAR(40)	NULL	SENIOR/PWD/MANUAL + ID no. (BR-009)
order.vatable_sales / vat_exempt_sales / vat_amount	NUMERIC(12,2)	NOT NULL	Tax breakdown (FR-103)
order.customer_name / phone / address	VARCHAR/TEXT	required for DELIVERY	FR-080
order.cancel_reason / cancelled_by	VARCHAR(255)/FK	required when CANCELLED	FR-086
order_item.menu_item_id	BIGINT	FK NOT NULL	
order_item.item_name / unit_price	VARCHAR / NUMERIC(12,2)	NOT NULL	Snapshot at sale (BR-007)
order_item.quantity	SMALLINT	CHECK ≥ 1	
order_item.notes	VARCHAR(255)		'No onions'
order_item_addon.addon_id / name / price	FK / VARCHAR / NUMERIC	NOT NULL	Snapshot

payment

Append-only settlement records.

Column	Type	Constraints	Description
--------	------	-------------	-------------

Column	Type	Constraints	Description
order_id	BIGINT	FK NOT NULL	Exactly one order (BR-005)
method	VARCHAR(14)	CHECK IN (CASH, GCASH, CARD, BANK_TRANSFER)	FR-100
amount	NUMERIC(12,2)	NOT NULL (negative = refund)	FR-105
tendered / change_given	NUMERIC(12,2)	NULL (cash only)	FR-101
reference_no	VARCHAR(60)	required for non-cash	
refund_of_id	BIGINT	FK→payment NULL	Links refund to original
received_by	BIGINT	FK→user NOT NULL	
receipt_no	BIGINT	UNIQUE(restaurant_id, receipt_no)	Serial receipt number (FR-103)
business_date	DATE	NOT NULL	Day locking

expense / employee / attendance

Operating cost & staff records.

Column	Type	Constraints	Description
expense.category	VARCHAR(20)	CHECK per FR-110	
expense.amount	NUMERIC(12,2)	CHECK > 0	
expense.expense_date	DATE	NOT NULL	Day locking (FR-113)
expense.receipt_key	VARCHAR(255)		Blob Storage key
expense.status	VARCHAR(10)	CHECK IN (PENDING, APPROVED)	FR-111
employee.full_name / position	VARCHAR	NOT NULL	
employee.daily_rate / monthly_salary	NUMERIC(12,2)	NULL; Owner/Manager visibility	FR-121
employee.user_id	BIGINT	FK→user NULL UNIQUE	Optional login link
attendance.employee_id / work_date	FK / DATE	UNIQUE together	
attendance.clock_in / clock_out	TIMESTAMPTZ		FR-122

daily_closing / notification / audit_log

Closing, alerts, and audit.

Column	Type	Constraints	Description
closing.business_date	DATE	UNIQUE(restaurant_id, business_date)	One closing per day
closing.opening_float / expected_cash / counted_cash / variance	NUMERIC(12,2)	NOT NULL	FR-140, FR-145
closing.variance_note	VARCHAR(255)	required if variance > 100	FR-141
closing.status	VARCHAR(10)	CHECK IN (CLOSED, REOPENED)	FR-144
closing.closed_by / reopened_by	FK→user		
notification.type / title / body	VARCHAR / VARCHAR / TEXT	NOT NULL	FR-150
notification.target_role / user_id	VARCHAR / FK NULL	role- or user-addressed	FR-151
notification.read_at	TIMESTAMPTZ	NULL	FR-152
audit_log.actor_id / role	FK / VARCHAR	NOT NULL	FR-161
audit_log.action / entity / entity_id	VARCHAR	NOT NULL	

Column	Type	Constraints	Description
audit_log.before / after	JSONB	NULL	Update snapshots
audit_log.ip / user_agent	INET / VARCHAR(255)		

10. API Specification

10.1 Conventions

- Base URL: <https://api.tapsi.ph/api/v1> — all endpoints below are relative to it.
- Authentication: Authorization: Bearer <access_token> on every endpoint except auth/register/reset.
- Content type: application/json; image uploads use multipart/form-data.
- Pagination: ?page=&page_size= (default 25, max 100); responses include count, next, previous, results.
- Filtering: documented query params per resource; dates as YYYY-MM-DD (Asia/Manila business dates).
- Errors: RFC 9457 problem-style JSON: {"code": "INSUFFICIENT_STOCK", "detail": "...", "errors": {field: [msg]}}.
- Status codes: 200/201 success, 400 validation, 401 unauthenticated, 403 forbidden, 404 not found (or cross-tenant), 409 conflict (state machine), 423 locked (closed day), 429 rate-limited.
- Idempotency: POST /orders and POST /orders/{id}/payments accept an Idempotency-Key header (NFR-014).

10.2 Authentication

Method	Path	Description	Roles
POST	/auth/login	Obtain access + refresh tokens	All
POST	/auth/refresh	Rotate refresh, new access token	All
POST	/auth/logout	Blacklist refresh token	All
POST	/auth/password/forgot	Send reset email	Public
POST	/auth/password/reset	Reset via emailed token	Public
POST	/auth/password/change	Change own password	All
GET/PATCH	/auth/me	View / update own profile	All

10.3 Restaurant & Users

Method	Path	Description	Roles
POST	/register	Register restaurant + Owner (public)	Public
GET/PATCH	/restaurant	View / update tenant settings	O (view: O,M)
GET/POST	/users	List / create staff users	O, M*
GET/PATCH	/users/{id}	View / edit user, change role	O (M limited)
POST	/users/{id}/deactivate	Deactivate user	O
DELETE	/users/{id}	Soft-delete user	O

10.4 Menu & Recipes

Method	Path	Description	Roles
GET/POST	/categories	List / create categories	O, M (list: all)
PATCH/DELETE	/categories/{id}	Update / soft-delete	O, M
GET/POST	/menu-items	List (filter: category, q, available) / create	O, M (list: all)
GET/PATCH/DELETE	/menu-items/{id}	Detail / update / soft-delete	O, M
POST	/menu-items/{id}/image	Upload image (multipart)	O, M
POST	/menu-items/{id}/availability	Toggle availability	O, M
GET	/menu-items/{id}/price-history	Price change log	O, M
GET/PUT	/menu-items/{id}/recipe	View / replace recipe lines	O, M

Method	Path	Description	Roles
GET/POST	/addons • PATCH /addons/{id}	Manage add-ons	O, M

10.5 Inventory, Suppliers & Purchasing

Method	Path	Description	Roles
GET/POST	/ingredients	List (filter: low_stock=true, q) / create	O, M (list: all)
GET/PATCH	/ingredients/{id}	Detail / update master data	O, M
GET	/ingredients/{id}/transactions	Stock card ledger (filter: type, date range)	O, M
POST	/inventory/adjustments	ADJUSTMENT / SPOILAGE / RETURN entry	O, M
GET/POST	/suppliers • GET/PATCH /suppliers/{id}	Supplier CRUD + history	O, M
POST	/suppliers/{id}/payments	Record supplier payment	O, M
GET/POST	/purchase-orders	List (filter: status, supplier) / create draft	O, M
GET/PATCH	/purchase-orders/{id}	Detail / edit while DRAFT	O, M
POST	/purchase-orders/{id}/place	DRAFT → ORDERED	O, M
POST	/purchase-orders/{id}/receive	Receive quantities (partial ok)	O, M
POST	/purchase-orders/{id}/cancel	Cancel (DRAFT/ORDERED only)	O, M

10.6 Orders, Tables & Kitchen

Method	Path	Description	Roles
GET/POST	/orders	List (filter: status, type, date, table) / create	O, M, C
GET/PATCH	/orders/{id}	Detail / edit items while PENDING	O, M, C
POST	/orders/{id}/status	Advance status (kitchen flow)	O, M, K
POST	/orders/{id}/discount	Apply Senior/PWD/manual discount	O, M, C*
POST	/orders/{id}/cancel	Cancel with reason (restores stock)	O, M
POST	/orders/{id}/complete	Verify payment, deduct stock, complete	O, M, C
GET	/kitchen/queue	PENDING + PREPARING oldest-first	O, M, K
GET/POST	/tables • PATCH /tables/{id}	Table CRUD	O, M (status: +C)
POST	/tables/{id}/status	Set AVAILABLE/.../CLEANING	O, M, C
POST	/orders/{id}/transfer	Move order to another table	O, M, C
POST	/orders/{id}/merge	Merge another open order into this one	O, M, C

10.7 Payments, Expenses & Closing

Method	Path	Description	Roles
POST	/orders/{id}/payments	Record payment (idempotent)	O, M, C
GET	/orders/{id}/receipt	Receipt payload (print view)	O, M, C
POST	/payments/{id}/refund	Refund with reason	O, M
GET/POST	/expenses	List (filter: category, date, status) / create	O, M, C (create)
PATCH/DELETE	/expenses/{id}	Edit / delete (open days only)	O, M
POST	/expenses/{id}/approve	Approve over-threshold expense	O, M
GET/POST	/employees • GET/PATCH /employees/{id}	Employee CRUD	O, M
POST	/attendance/clock	Clock in/out	All
GET	/attendance?month=	Monthly summary	O, M
GET	/closings?date=	Closing status / preview expected cash	O, M
POST	/closings	Perform daily closing	O, M
POST	/closings/{id}/reopen	Reopen closed day with reason	O

10.8 Dashboard, Reports, Notifications & Audit

Method	Path	Description	Roles
GET	/dashboard	Role-aware widget payload	O, M, C
GET	/reports/daily-sales?date=	Daily sales report	O, M

Method	Path	Description	Roles
GET	/reports/monthly-sales?month=	Monthly report	O, M
GET	/reports/profit-loss?from=&to=	P&L	O, M
GET	/reports/inventory-usage?from=&to=	Usage & spoilage	O, M
GET	/reports/product-mix?from=&to=	Best/least sellers	O, M
GET	/reports/purchases?from=&to=	Purchase report	O, M
GET	/reports/tax-summary?from=&to=	VAT summary	O, M
POST	/reports/{name}/export	Async CSV/PDF export job	O, M
GET	/notifications • POST /notifications/read-all	List / mark read	All
POST	/notifications/{id}/read	Mark one read	All
GET	/audit-logs	Search (user, entity, action, range)	O (M read)

Roles legend: O = Owner, M = Manager, C = Cashier, K = Kitchen. Asterisk (*) = restricted variant per Section 3.2 (e.g., Manager creates only Cashier/Kitchen users; Cashier discounts above 10% need approval).

10.9 Representative Payloads

POST /orders — request:

```
{
  "type": "DINE_IN",
  "table_id": 3,
  "items": [
    {"menu_item_id": 11, "quantity": 2, "notes": "no onions",
     "addons": [{"addon_id": 4}]},
    {"menu_item_id": 15, "quantity": 1}
  ]
}
```

201 response (abridged):

```
{
  "id": 982, "reference": "5f0c...", "order_no": 42, "status": "PENDING",
  "business_date": "2026-07-18", "subtotal": "465.00",
  "discount_amount": "0.00", "total": "465.00",
  "vatable_sales": "415.18", "vat_amount": "49.82", "vat_exempt_sales": "0.00",
  "items": [ {"item_name": "Tapsilog", "unit_price": "165.00",
    "quantity": 2, "addons": [{"name": "Extra Rice", "price": "20.00"}]} ]
}
```

409 on POST /orders/{id}/complete when stock is insufficient:

```
{
  "code": "INSUFFICIENT_STOCK",
  "detail": "Cannot complete order #42: stock would go negative.",
  "errors": {"ingredients": [
    {"id": 7, "name": "Beef Tapa", "required": "0.400", "available": "0.250", "unit": "kg"}
  ]}
}
```

11. UI / UX Requirements

11.1 General

- Responsive layout: desktop (≥1280px) for back-office screens; tablet (≥768px) optimized for the order, kitchen, and table screens.
- Global navigation shows only modules permitted for the current role (Section 3.2).
- Every list screen provides search, filters, column sort, pagination, and an empty state with a primary call to action.
- All forms validate inline on blur and on submit; server validation errors map to the offending fields.
- Loading states use skeletons; destructive actions use confirmation dialogs naming the object (NFR-044).
- Currency inputs accept digits and a single decimal point; display format ₱1,234.50.

11.2 Key Screens

Screen	Primary Role	Content / Behavior
Login / Reset	All	Email + password; forgot-password flow; lockout messaging
Dashboard	Owner/Manager	KPI cards, order-status board, top sellers, low stock, 30-day trend
Order Screen (POS)	Cashier	Left: category tabs + searchable menu grid with images; Right: cart with quantities, notes, add-ons, discounts, running total; large touch targets
Kitchen Queue	Kitchen	Card per order, oldest first, elapsed-time badge, one-tap status advance; auto-refresh ≤5 s
Table Map	Cashier	Color-coded grid (green available, red occupied, yellow reserved, gray cleaning) with running bill; tap to open/create order
Payment Modal	Cashier	Method tabs, tendered/change calculator, split payment rows, complete-order button gated on full settlement
Receipt View	Cashier	58/80mm print stylesheet; REPRINT watermark on reprints
Inventory / Stock Card	Manager	Ingredient list with low-stock highlighting; per-ingredient ledger with running balance
Purchase Orders	Manager	PO list by status; receive dialog with per-line quantities
Reports	Owner	Date-range picker, tabular results with totals row, chart where applicable, CSV/PDF export
Daily Closing Wizard	Owner/Manager	Stepper: open orders check → expected cash → count entry → variance → confirmation
Settings / Users	Owner	Restaurant profile, receipt fields, user management, thresholds

12. Security Requirements (Consolidated)

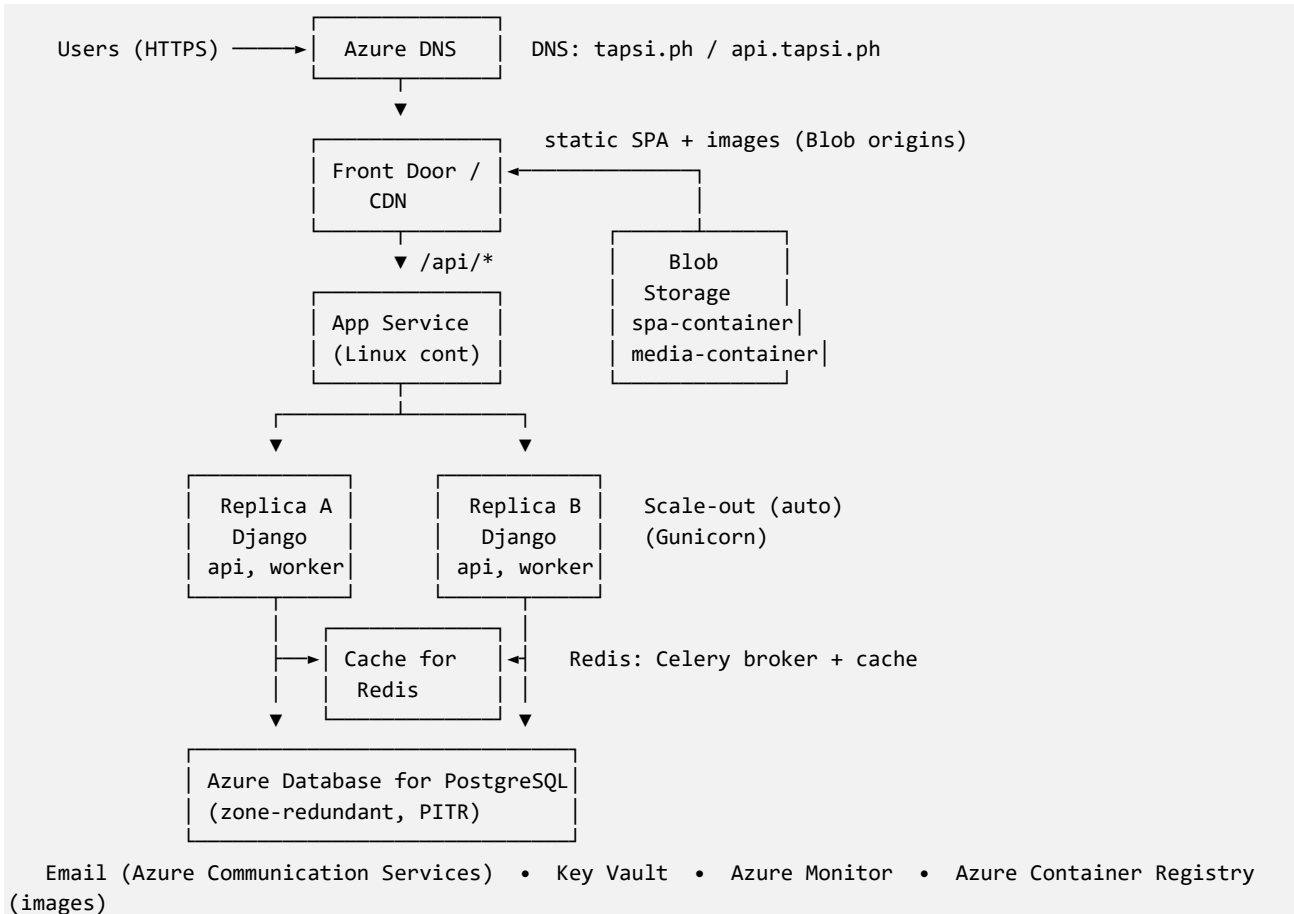
Security NFRs are specified in Section 7.3; this section consolidates the security architecture for review.

- Authentication: JWT access (15 min) + rotating refresh (7 days) with server-side blacklist on logout/password change; account lockout (FR-003).
- Authorization: DRF permission classes per role + object-level tenant checks on every retrieve/update (NFR-020); permission-denied attempts audit-logged (FR-160).
- Transport: TLS 1.2+ everywhere; HSTS max-age ≥ 6 months; Azure Front Door redirects HTTP→HTTPS.
- Data at rest: Azure Database for PostgreSQL and Blob Storage encryption (AES-256); Azure Key Vault for credentials with rotation.
- Input handling: DRF serializer validation; ORM-parameterized queries; file uploads validated for type, size, and image integrity; uploaded files never executed or served from the app domain.
- Headers: Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, frame-ancestors 'none'.
- Rate limiting per NFR-024 at the API gateway/application layer; login throttling per IP and per account.

- Auditability: append-only audit log (Module 17) covering authentication, data changes, and privileged actions.
- Privacy: DPA 2012 compliance — minimal collection, role-restricted access to employee salary and customer contact data, erasure workflow via support.
- Software supply chain: dependency pinning, Dependabot updates, image scanning in CI (NFR-028); production images run as non-root.

13. Azure Architecture & Deployment

13.1 Target Architecture



13.2 Environments & CI/CD

Stage	Trigger	Actions
CI (every PR)	Pull request	Lint (ruff, eslint) → unit tests + coverage gate (NFR-050) → build Docker image → dependency & image scan
Staging deploy	Merge to main	Push image to Azure Container Registry → run migrations → deploy to staging App Service → smoke tests (login, create order, complete payment)
Production deploy	Git tag vX.Y.Z (manual approval)	Blue/green: new deployment slot receives traffic after health checks; automatic rollback on failed health or elevated 5xx

- Migrations run before traffic shift and must be backward compatible with the previous release (expand–migrate–contract).

- Infrastructure defined as code (Terraform with the AzureRM provider) and versioned alongside the application.
- Environments: local (Docker Compose with Postgres + Redis + Azurite), staging (scaled-down copy), production.

14. Backup & Disaster Recovery

Item	Specification
Azure Database for PostgreSQL automated backups	Daily snapshots, 30-day retention; point-in-time recovery enabled (5-min granularity)
Blob Storage media	Versioning enabled; lifecycle to cool tier after 90 days; geo-redundant storage (GRS) for the media container
Recovery Point Objective (RPO)	≤ 5 minutes (PITR)
Recovery Time Objective (RTO)	≤ 4 hours for full region-internal recovery
Restore drills	Quarterly: restore latest snapshot to staging and run the smoke suite; results documented
Configuration	Terraform state and Azure Key Vault allow full environment rebuild
Data export	Owners may request a full export of their tenant data (CSV bundle) — supports DPA portability

15. Logging & Monitoring

- Structured JSON application logs with request ID, tenant ID, user ID, route, latency, and status; shipped to Azure Monitor Logs (30-day hot retention, 1-year archive to Blob Storage).
- Metrics: request rate, p95/p99 latency, error rate, queue depth (Celery), DB connections, replication lag — Azure Monitor dashboards per environment.
- Alarms (to email/Slack): 5xx rate > 1% for 5 min; p95 latency > 800 ms for 10 min; App Service scale-out at max capacity; Azure Database for PostgreSQL storage > 80%; failed Celery tasks; failed backups.
- Error tracking: Sentry (or equivalent) for exception aggregation with release tagging.
- Health endpoints: /healthz (liveness) and /readyz (DB + Redis checks) consumed by Azure Front Door and App Service health probes.
- Never logged: passwords, tokens, full card/GCash reference PANs, or request bodies of auth endpoints.

16. Testing Requirements & Acceptance

16.1 Test Levels

Level	Scope	Tooling	Gate
Unit	Pricing service, discount/VAT math, stock deduction, state machines	pytest + factory_boy	≥80% coverage on business logic (NFR-050)
API / Integration	Every endpoint: happy path, validation, authZ (role × endpoint matrix), tenant isolation	pytest + DRF test client, real Postgres in CI	All FR acceptance criteria automated
Concurrency	Simultaneous order completion vs. stock; double-submit payments; concurrent PO receive	pytest with threads / transactional tests	No negative stock, no duplicate payments
E2E	UC-01...UC-08 through the real UI	Playwright	Green on staging before production deploy
Performance	NFR-001/003 under 100 concurrent users, 30-min soak	k6 or Locust	p95 < 300 ms on CRUD
Security	AuthZ matrix, IDOR probes, rate limits, OWASP baseline	OWASP ZAP baseline + targeted tests	No high-severity findings

16.2 Mandatory Test Scenarios

- Tenant isolation: user of restaurant A requests every resource type of restaurant B by ID → 404, and the attempt is audit-logged.
- Order completion deducts exact recipe quantities (items + add-ons) atomically; a forced failure mid-transaction leaves no partial state (NFR-012).
- Completing an order with insufficient stock fails with INSUFFICIENT_STOCK listing shortfalls; nothing is deducted (BR-001).
- Cancelling before vs. after consumption restores stock correctly (BR-003).
- Two cashiers completing the last portion concurrently: exactly one succeeds (row locking).
- Senior discount: 20% off, VAT-exempt classification, ID captured, correct receipt tax block (BR-009).
- Split payment (GCash + cash) settles order; completion blocked while any balance remains (FR-102).
- Day close then attempted back-dated expense/order/payment → 423 Locked (BR-008); Owner reopen path works and is audit-logged.
- P&L over a range equals the sum of its daily components and reproduces identically on re-run (BR-013).
- Idempotency: retried POST /orders with the same Idempotency-Key returns the original order, not a duplicate (NFR-014).

16.3 Release Acceptance Criteria (Definition of Done for v1.0)

- All Must (M) functional requirements implemented with automated tests passing.
- All user stories in Section 5 pass their acceptance criteria on staging.
- Performance targets (NFR-001, NFR-003) demonstrated by a recorded load-test run.
- Security checklist of Section 12 verified; ZAP baseline scan clean of high findings.
- A pilot restaurant completes 3 consecutive real business days — including daily closings — without data corrections.
- Runbooks exist for deploy, rollback, restore-from-backup, and day-reopen support cases.

17. Future Enhancements (Version 2+)

Candidate	Notes
Kitchen Display System (KDS)	Dedicated always-on kitchen screen with bump bar support
QR code table ordering	Customers order from their phones; feeds the same order pipeline
Online ordering & delivery integrations	GrabFood / foodpanda menu sync and order ingestion
Customer loyalty, coupons, gift cards	Requires customer identity model
SMS notifications	Order-ready and marketing (opt-in) via local SMS gateway
Inventory forecasting & AI sales prediction	Reorder suggestions from consumption history
Accounting integration	Xero/QuickBooks journal export of sales, VAT, and expenses
Payroll	From attendance + salary data already captured in v1.0
Multi-branch	Branch entity above restaurant; consolidated reporting; stock transfers
Offline mode	Local-first POS with sync/conflict resolution
Native mobile app	Owner dashboard and approvals on the go

Appendix A — Glossary

Term	Definition
Add-on	An optional paid extra attached to a menu item (e.g., extra rice)
Azure Front Door	Azure Front Door
BIR	Bureau of Internal Revenue (Philippine tax authority)
Business day	The Asia/Manila calendar date an order/payment/expense belongs to (BR-014)
COGS	Cost of goods sold — derived from CONSUMPTION transactions valued at weighted average cost
Daily closing	End-of-day cash reconciliation that locks the business day
DPA	Data Privacy Act of 2012 (RA 10173)
DRF	Django REST Framework
IDOR	Insecure Direct Object Reference — accessing another tenant's records by guessing IDs
JWT	JSON Web Token, used for stateless API authentication
Karinderya	Small Filipino eatery serving pre-cooked meals
MoSCoW	Prioritization scheme: Must, Should, Could, Won't
Opening float	Cash placed in the drawer at the start of the business day
Paresan	Eatery specializing in pares (braised beef stew with rice/noodles)
PITR	Point-in-time recovery (database restore to a specific moment)
PO	Purchase order
PWD	Person with Disability (RA 10754 20% discount + VAT exemption)
Recipe	Bill of materials mapping a menu item to ingredient quantities
RBAC	Role-based access control
Senior Citizen discount	Mandatory 20% discount + VAT exemption under RA 9994
Silog	Rice meal combining garlic rice (sinangag) and egg (itlog), e.g., tapsilog
Tapsilogan	Eatery specializing in silog meals
Tenant	One restaurant and all its isolated data within the SaaS platform
TIN	Tax Identification Number
VAT	Value-added tax (12% in the Philippines)
Weighted average cost	Inventory costing: total cost of stock on hand ÷ quantity on hand
X-Reading / Z-Reading	Mid-shift (non-resetting) and end-of-day sales summary reports

Appendix B — Sample Receipt Layout

BENJIE'S TAPSILOGAN
 123 Mabini St., Quezon City
 TIN: 123-456-789-000 VAT REG

OR No: 000001842 2026-07-18 12:41
 Cashier: Maria D. Table T3

2	Tapsilog	@165.00	330.00
	+ Extra Rice	@20.00	40.00
1	Iced Tea (L)	@55.00	55.00

Subtotal	425.00
Senior Disc (SC-1234567)	- 85.00

TOTAL	340.00
CASH	500.00
CHANGE	160.00

VATable Sales	0.00
VAT-Exempt Sales	340.00
VAT (12%)	0.00

Salamat po! Balik kayo!
THIS SERVES AS YOUR OFFICIAL
SALES INVOICE

Appendix C — Application Error Codes

Code	HTTP	Meaning
VALIDATION_ERROR	400	One or more fields failed validation; see errors map
INVALID_CREDENTIALS	401	Login failed (generic; no account enumeration)
ACCOUNT_LOCKED	401	Too many failed attempts; retry after lockout window
ACCOUNT_DISABLED	401	User is deactivated
TOKEN_EXPIRED / TOKEN_INVALID	401	Access token must be refreshed / is malformed
PERMISSION_DENIED	403	Role lacks permission for this action
NOT_FOUND	404	Resource missing or belongs to another tenant
STATE_CONFLICT	409	Illegal status transition (order, PO, table)
ORDER_IMMUTABLE	409	Attempt to modify a COMPLETED order (BR-002)
INSUFFICIENT_STOCK	409	Operation would drive ingredient stock negative (BR-001)
UNSETTLED_BALANCE	409	Completion attempted before full payment (FR-102)
DUPLICATE_REQUEST	409	Idempotency key already used with a different body
DAY_LOCKED	423	Business day is closed (BR-008)
RATE_LIMITED	429	Too many requests; see Retry-After header
SERVER_ERROR	500	Unexpected error; request ID included for support

Appendix D — Requirements Traceability (Roadmap)

Build order for the development roadmap, with the requirements each phase delivers. A phase is complete when its requirements' tests pass and its user stories meet acceptance criteria.

Phase	Deliverable	Requirements	Key Stories
1	Authentication & roles	FR-001..009, NFR-020..024	US-001, US-002
2	Restaurant & user management	FR-010..017	US-003, US-004
3	Menu & categories	FR-030..037	US-010, US-011
4	Inventory & suppliers	FR-040..053	US-020, US-021, US-022
5	Purchasing & recipes	FR-060..075	US-012, US-023
6	Orders, tables & kitchen	FR-080..095	US-030..US-034
7	Payments & receipts	FR-100..106	US-040..US-042
8	Expenses, employees, closing	FR-110..145	US-050..US-052
9	Dashboard, reports, notifications, audit	FR-020..026, FR-130..163	US-060..US-062
10	Hardening & Microsoft Azure production deployment	All NFRs, Sections 13–16	Release acceptance (16.3)